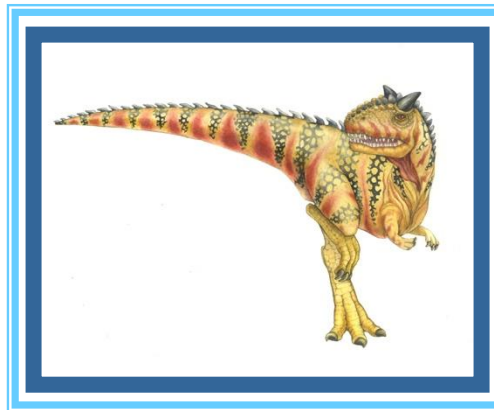
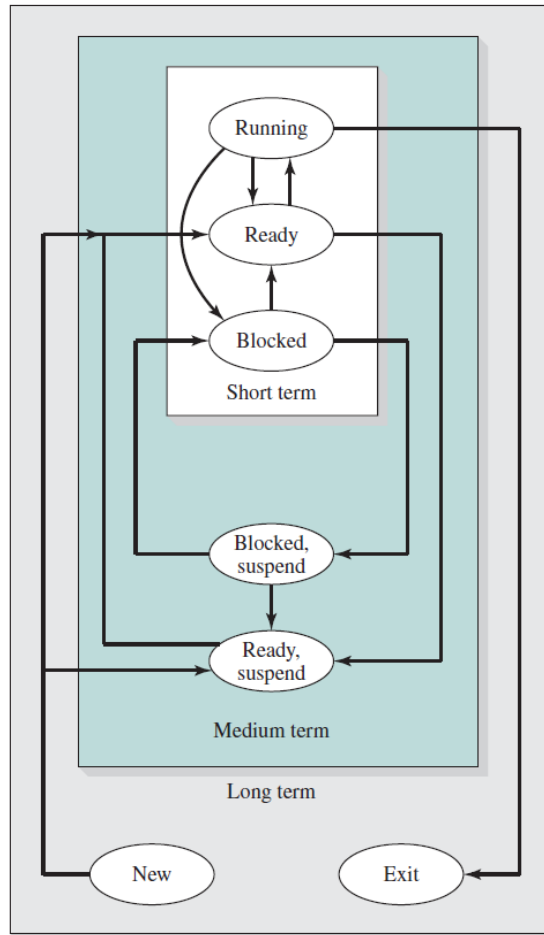


Calendarización de procesos





Calendarizadores



- **Calendarización**, en general, se refiere a decidir a qué proceso asignar un recurso.
- La calendarización se suele llevar a cabo en dos o tres niveles:
 - Calendarización a corto plazo (**CPU scheduling**).
 - Calendarización a mediano plazo (**calendarización de memoria**).
 - Calendarización a largo plazo (**calendarización de admisiones**).





Calendarización a largo plazo

- ❑ Llevada a cabo por el **calendarizador de admisiones** o **job scheduler**.
- ❑ Característica de *batch systems*. No son interactivos (*i.e.*, trabajan sin intervención del usuario).
 - ❑ En un *batch system* se ingresan solicitudes de trabajo que el sistema primero admite y luego encola.
- ❑ Ante una solicitud admitida se crea un proceso que compite por CPU.





Calendarización a corto y mediano plazo

- CP: llevada a cabo en sistemas *batch* e interactivos, por el **calendarizador de CPU**.
 - Decide qué procesos en la **ready queue** (en memoria principal) usan un procesador.

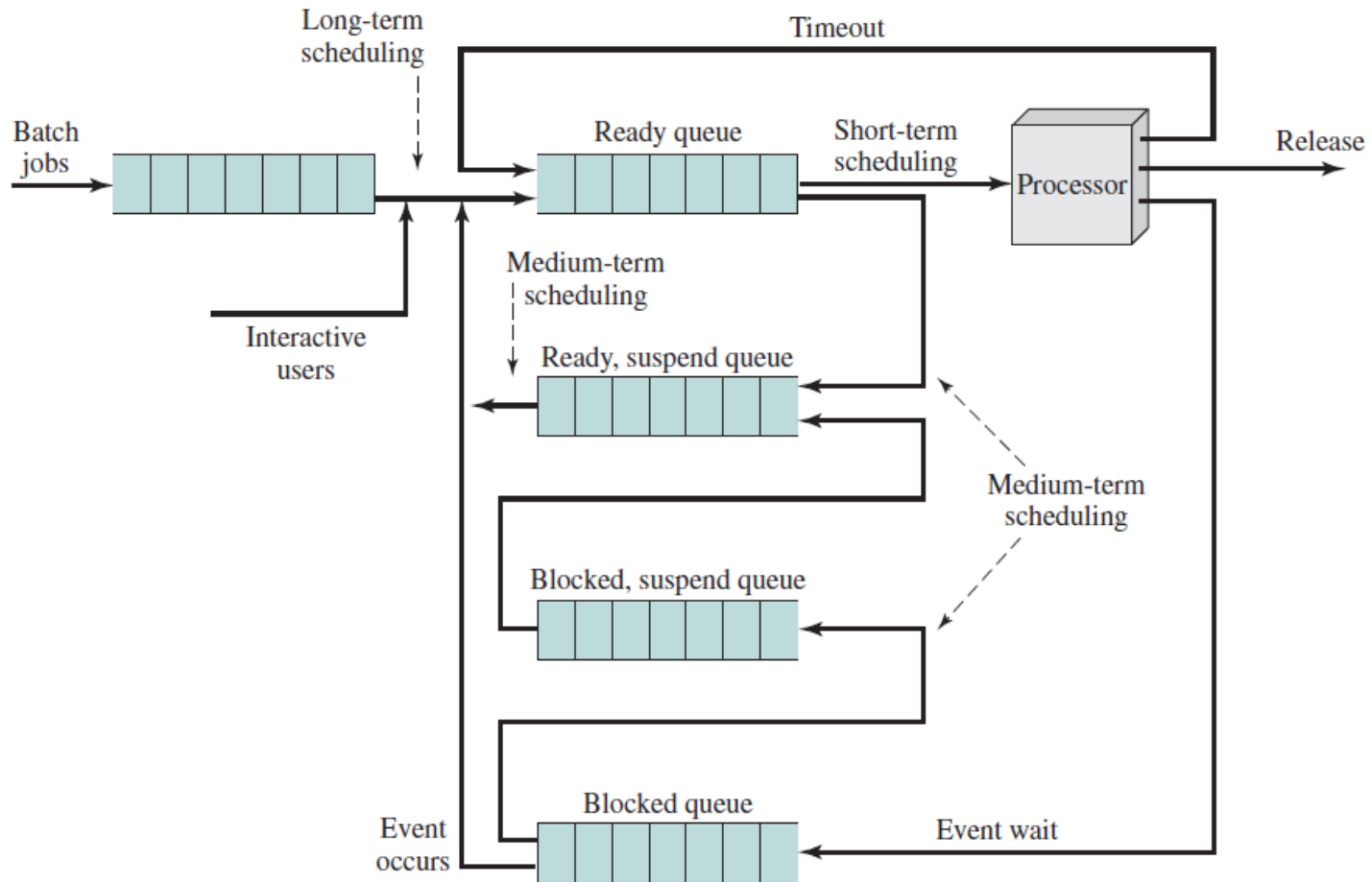


- MP: llevada a cabo en sistemas *batch* e interactivos por el **calendarizador de memoria**.
 - Controla el número de procesos en memoria principal (*i.e.* en la *ready queue*).
 - ▶ Este número es llamado **grado de multiprogramación**.





Calendarización de tres niveles





Ambientes de operación

- Los diferentes tipos de calendarizador surgen por restricciones de espacio y tiempo.
 - Para los diferentes tiempos de operación de los procesos, las colas adaptan la atención (*i.e.*, recursos) que el sistema les provee.
 - Podríamos tener más procesos pendientes de ejecución que memoria principal (o restringir la memoria usada para encolar procesos).
- Los calendarizadores (algoritmos de calendarización) buscan cumplir diferentes objetivos según el ambiente en el que se usen.
 - **Batch systems**: maximizar tareas completadas por tiempo, y uso del procesador.
 - **Sistemas interactivos**: minimizar tiempo de respuesta y proveer proporcionalidad.
 - **Sistemas de tiempo real**: previsibilidad y cumplimiento de cotas para el tiempo de ejecución de tareas.





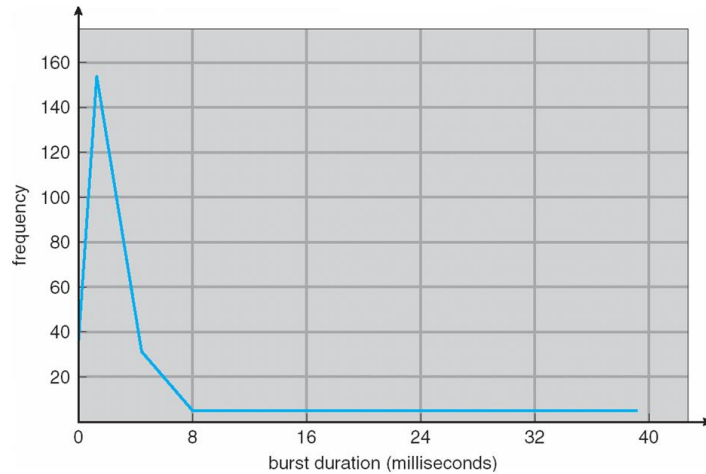
Comportamiento de los procesos

- La ejecución de procesos suele alternar entre ejecución en CPU y espera por I/O (u otro proceso).
- Los momentos de ejecución son llamados **CPU burst** y los de espera son llamados **I/O burst**.
 - Algunas actividades de I/O cuentan como tiempo de computación porque el CPU está trabajando.
- Cuando sus *bursts* de CPU son más largos que los de I/O, el proceso es **CPU** o **compute-bound**. En el caso inverso, son **I/O-bound**.
 - Identificar procesos como CPU o I/O *bound* require uso de herramientas y/o conocimiento del código del proceso.
 - Esta información no suele estar disponible para el calendarizador de CPU, por lo que se hacen predicciones basadas en *bursts* ya realizados.



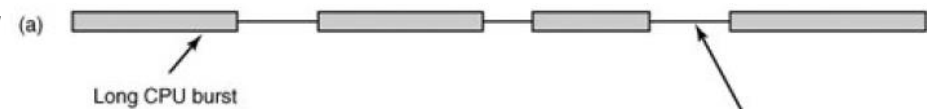


Comportamiento de los procesos

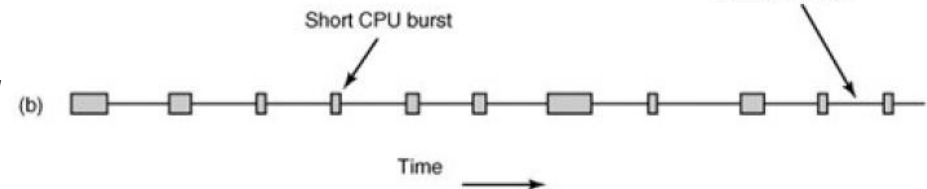


Histograma habitual de frecuencia contra duración de CPU *bursts*.

CPU-bound



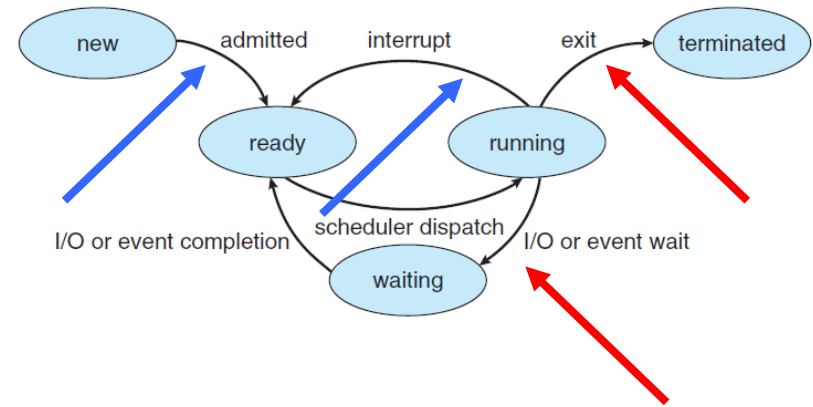
I/O-bound





Decisiones de calendarización

- Cuando un proceso termina y cuando se bloquea.
 - El estado del proceso pasa de ejecutándose a bloqueado.
 - Cuando el proceso es desbloqueado entra de nuevo a la *ready queue* (i.e., pasa de bloqueado a listo).
- Cuando un proceso nuevo es creado o cuando ocurre una interrupción (de I/O o de reloj).
 - El estado del proceso pasa de ejecutándose directamente a listo.





Decisiones de calendarización

- Si se toman decisiones cuando un proceso es creado y/o cuando ocurre una interrupción, la calendarización es **preemptive**.
 - De lo contrario es **non-preemptive** o **cooperativa**.
 - Se refiere a la posibilidad de interrumpir un proceso en ejecución.
- Este tipo de calendarización es esencial para sistemas interactivos.





Cómo tomar las decisiones

- Todo sistema debe proveer una porción justa del CPU a cada proceso.
 - Los procesos pueden ser categorizados por el sistema y ser tratados justamente dentro de sus respectivas categorías.
- Las políticas del sistema deben ser cumplidas por el calendarizador.
 - Las **políticas** determinan qué se hace para lograr un objetivo del sistema; los **mecanismos** determinan cómo se hace.
- Todas las partes del sistema se deben mantener ocupadas. Esto involucra mantener en memoria una buena mezcla de procesos.
- Objetivos adicionales dependen del ambiente de operación (*batch*, interactivo o tiempo real).





Criterios de calendarización de CPU

- ❑ **Tiempo de respuesta** –
- ❑ **Tiempo de completación** –
- ❑ **Previsibilidad** –





Criterios de calendarización de CPU

- **Tiempo de respuesta** – tiempo desde que se somete una solicitud hasta que se recibe la primera respuesta del sistema.
 - Desde la perspectiva de un proceso.
 - Importante medida en sistemas interactivos por ser orientada a usuarios.
- **Tiempo de completación** – tiempo desde la solicitud de ejecución de un proceso hasta su terminación.
 - Orientado a usuarios, pero más importante en *batch systems*.
- **Previsibilidad** – la carga en el sistema no debería afectar el tiempo que toma completar una misma tarea.
 - Orientada a usuarios.
 - Suele ser importante en algunos sistemas de tiempo real.





Criterios de calendarización de CPU

- **Uso del CPU** –
- ***Throughput*** –
- **Tiempo de espera** –





Criterios de calendarización de CPU

- **Uso del CPU** – mantener al procesador tan ocupado como sea posible.
 - La unidad de medida es porcentaje de tiempo que el procesador está computando.
 - Orientado al sistema.
- **Throughput** – número de procesos completados por unidad de tiempo.
 - Orientado al sistema.
 - ▶ A un usuario no le interesa medir qué tanto trabajo está haciendo la máquina sino qué tanto tiempo le toma obtener una respuesta para su solicitud.
- **Tiempo de espera** – tiempo que un proceso pasa en la *ready queue*.
 - Orientado al sistema, pero relacionado con usabilidad.
 - Un concepto similar es **fairness**: ningún proceso debería sufrir *starvation*.
 - ▶ **Starvation** (inanición) sucede cuando un proceso pasa un tiempo indefinido en la *ready queue* por priorización de otros procesos.





ALGORITMOS DE CALENDARIZACIÓN





First-Come, First-Served (FCFS)

- Los procesos siguen una cola FIFO.
 - Es el algoritmo más sencillo.
 - Es justo: todos los procesos reciben el procesador eventualmente (si ningún proceso en ejecución se encierra en un ciclo infinito).
- Cuantitativamente, la **función de selección** otorga el procesador al proceso que lleve más tiempo en la *ready queue*.
 - Favorece a procesos largos por otorgarles el procesador por el tiempo que quieran.
 - Favorece a los procesos *CPU-bound* porque los *I/O bound* salen rápido del procesador y los *CPU-bound* se atiborran.
- *Non-preemptive*.





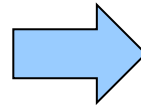
Ejemplo FCFS

Consideremos los procesos en la tabla.

Compárense los diagramas para la ejecución de cada proceso cuando llegan en los siguientes órdenes, respectivamente:

- P_1, P_2, P_3
- P_2, P_3, P_1

Silberschatz llama a esto **efecto convoy**.



Process	Burst Time
P_1	24
P_2	3
P_3	3



Tiempo de espera para $P_1 =$; $P_2 =$
; $P_3 =$
Tiempo de espera promedio:



Tiempo de espera para $P_1 =$; $P_2 =$
; $P_3 =$
Tiempo de espera promedio:





Shortest-Job-First (SJF)

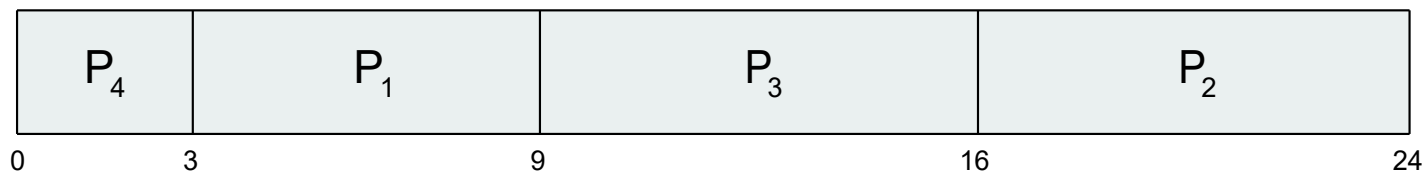
- Supone que se pueden conocer los tiempos de ejecución de cada proceso con anticipación.
- Funciona si los procesos están todos disponibles a la vez (es decir, que no llegan a la cola en diferentes momentos).
 - De lo contrario, procesos largos corren riesgo de *starvation*.
- Si se cumplen estas condiciones, SJF es óptimo.
- Contrarresta el favorecimiento de procesos CPU-bound y mejora tiempos de respuesta.
- *Non-preemptive*.





Ejemplo SJF

<u>Proceso</u>	<u>Burst</u>
P_1	6
P_2	8
P_3	7
P_4	3



□ Tiempo de espera promedio =





Shortest-Remaining-Time (SRT)

- Es versión *preemptive* de SJF.
- Sufre las mismas desventajas.
- Mejora el tiempo de completación.

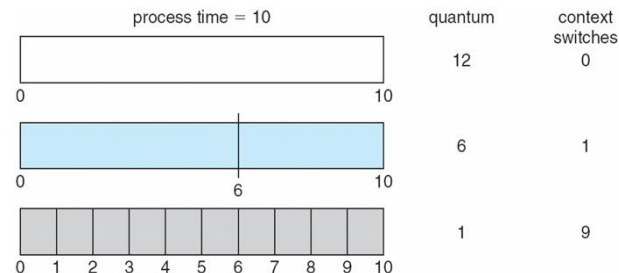
Process	A	B	C	D	E	
Arrival Time	0	2	4	6	8	
Service Time (T_s)	3	6	4	5	2	Mean
SRT						
Finish Time	3	15	8	20	10	
Turnaround Time (T_r)	3	13	4	14	2	7.20
T_r/T_s	1.00	2.17	1.00	2.80	1.00	1.59
SJF SJF						
Finish Time	3	9	15	20	11	
Turnaround Time (T_r)	3	7	11	14	3	7.60
T_r/T_s	1.00	1.17	2.75	2.80	1.50	1.84
FCFS						
Finish Time	3	9	13	18	20	
Turnaround Time (T_r)	3	7	9	12	12	8.60
T_r/T_s	1.00	1.17	2.25	2.40	6.00	2.56





Round Robin (RR)

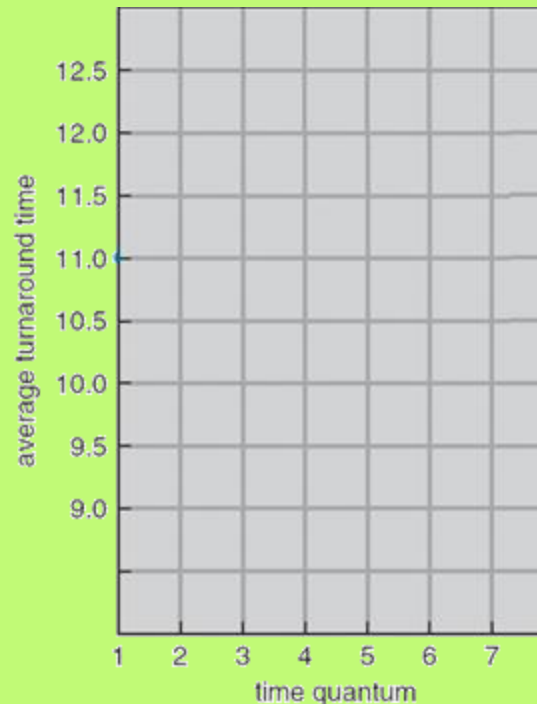
- Similar a FCFS, pero *preemptive*.
- Se define un **quantum de tiempo** que determina el ritmo con el que el calendarizador otorga el CPU a los procesos *ready*.
 - Se recorre la *ready queue* como una cola circular.
 - Si un proceso no se bloquea antes de que transcurra un *quantum*, el proceso es *preemptado* (reemplazado).
- El tamaño sí importa.
 - Un *quantum* demasiado grande degenera al algoritmo en un FCFS.
 - Un *quantum* muy pequeño ocasiona muchos cambios de contexto.
 - El *quantum* debería ser grande con respecto al tiempo que toma un cambio de contexto.





Round Robin (RR)

- Impone límites claros sobre el tiempo de CPU y de espera que tendrá cada proceso.
 - $\frac{1}{n}$ del tiempo de procesador, en proporción, para cada uno de n procesos.
 - $\leq q(n - 1)$ unidades de tiempo de espera por proceso, dado un *quantum* q .
- ¿Qué criterio de calendarización se ve principalmente afectado por el tamaño del *quantum*?
 - El tiempo de



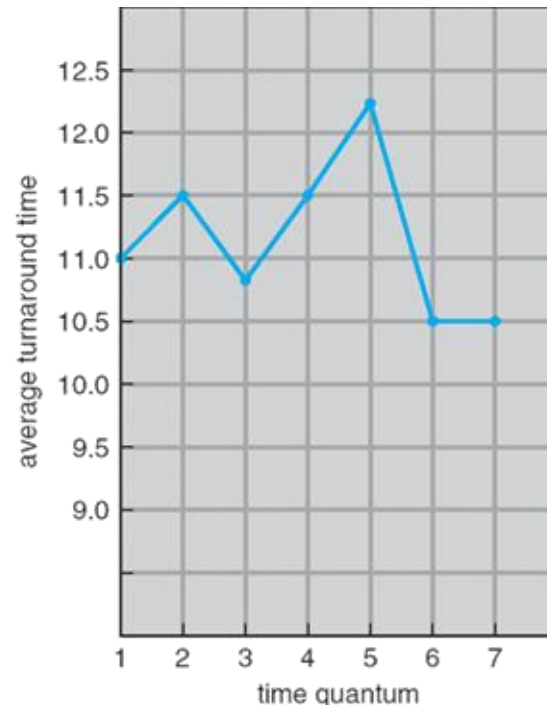
process	time
P_1	6
P_2	3
P_3	1
P_4	7





Round Robin (RR)

- Impone límites claros sobre el tiempo de CPU y de espera que tendrá cada proceso.
 - $\frac{1}{n}$ del tiempo de procesador, en proporción, para cada uno de n procesos.
 - $\leq q(n - 1)$ unidades de tiempo de espera por proceso, dado un *quantum* q .
- ¿Qué criterio de calendarización se ve principalmente afectado por el tamaño del *quantum*?
 - El tiempo de completación.



process	time
P_1	6
P_2	3
P_3	1
P_4	7





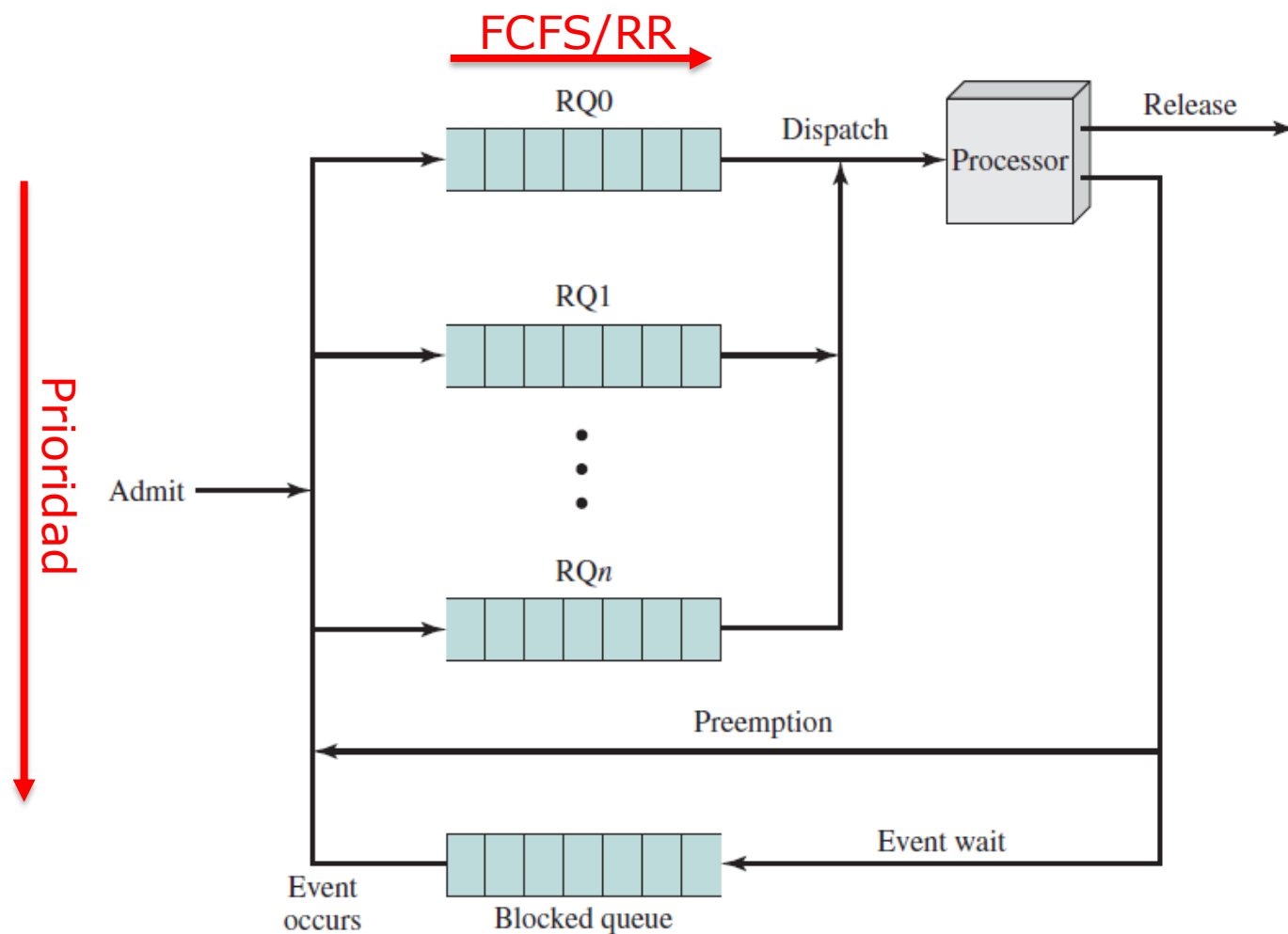
Calendarización por prioridades

- Es el caso general de asignación de prioridades (calculadas o provistas por usuario) a procesos.
 - SJF y SRT son casos especiales de este paradigma.
- Puede ser *preemptive* y *non-preemptive*.
- El paradigma conlleva el riesgo de *starvation*. Una solución a esto es el **envejecimiento de procesos**.
 - La prioridad de un proceso incrementa con el tiempo que pasa esperando el CPU.





Calendarización por prioridades





Cola multinivel

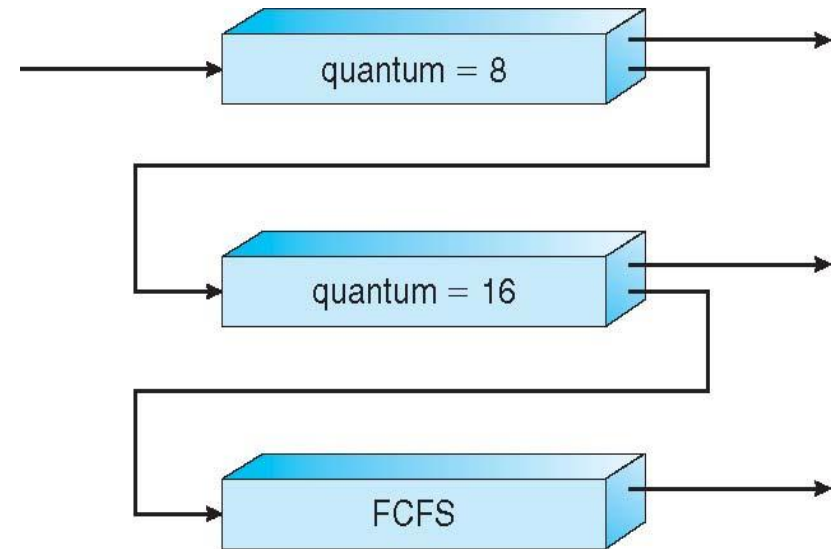
- Este algoritmo separa la *ready queue* en varias colas, cada una para una clase específica de proceso.
- Cada cola puede implementar un algoritmo de calendarización diferente.
- La atención a cada cola debe ser también calendarizada. El acercamiento habitual es colas con prioridades.
 - La variedad básica es que la prioridad asignada a los procesos en cada cola esté fija, lo que se llama **prioridades estáticas**.
 - Una variedad más flexible permite mover los procesos entre colas dependiendo de sus características y el tiempo que lleven en espera.
 - ▶ Esta variedad es llamada **cola multinivel con retroalimentación**.
- Sólo se ejecutan procesos de cierta prioridad cuando las colas con prioridades más altas están vacías.
 - Procesos que entran a prioridades altas *preemptan* el procesador sobre prioridades más bajas.





Cola multinivel

- Es un algoritmo muy general pero complicado.
 - Debemos determinar el mejor algoritmo por cola.
 - Debemos determinar el valor de todos los parámetros.
- El siguiente ejemplo consiste en bajar prioridad a procesos que excedan un cierto *quantum* por cola.
 - Q0 – RR con *quantum* de 8 ms.
 - Q1 – RR con *quantum* de 16 ms.
 - Q2 – FCFS.





Cola multinivel





Otros calendarizadores

Definamos el **calendarizador por lotería**

- Se reparten “*tickets*” de lotería. Una decisión de calendarización consiste en sortear el CPU entre los *tickets*.
- El sistema puede variar la cantidad de *tickets* que entrega a cada proceso para alterar prioridades.
- Los procesos pueden intercambiar *tickets*.

Definamos el **calendarizador *fair-share* (porción justa)**

- Considera a los usuarios propietarios de los procesos que compiten por CPU.
- Distribuye un porcentaje equitativo del tiempo de CPU a cada usuario.
 - Sin importar la cantidad de procesos que el usuario tenga, todos reciben la misma fracción de CPU.





Tabla comparativa

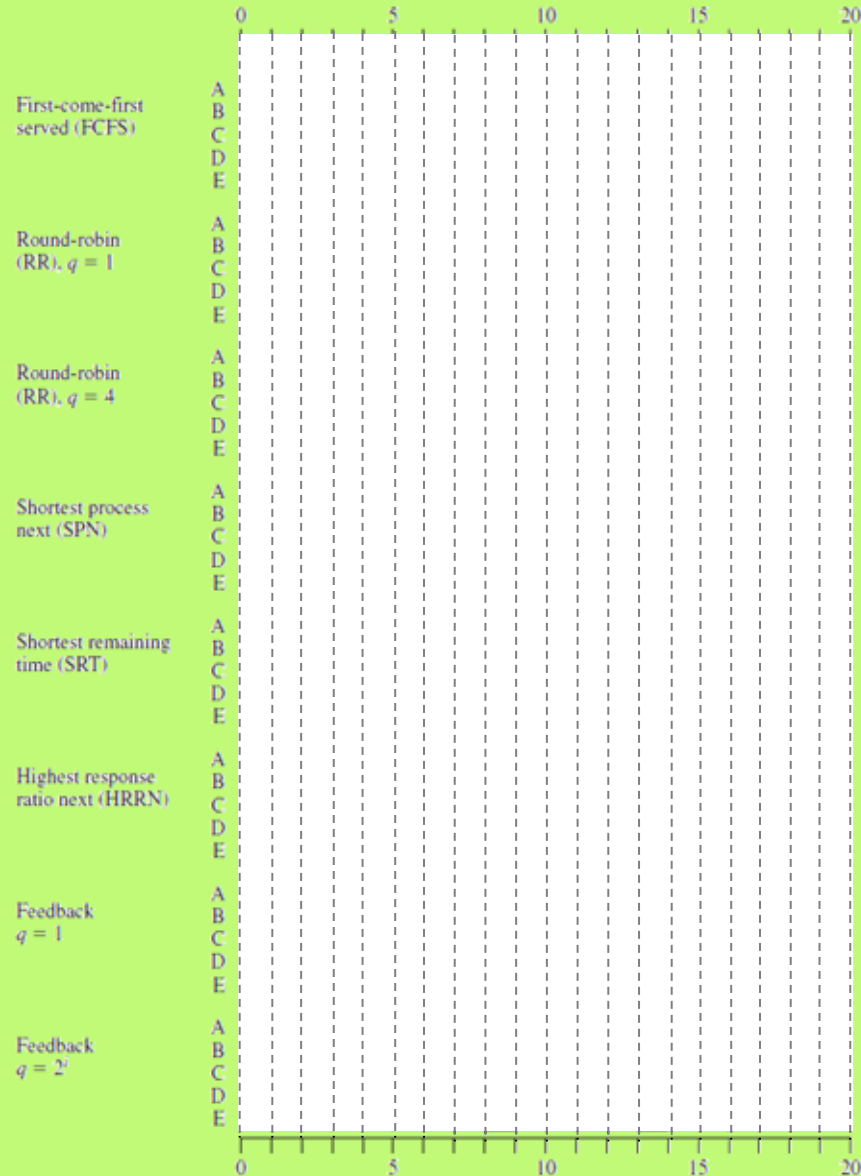
	FCFS	Round Robin	SPN	SRT	HRRN	Feedback
Selection Function	$\max[w]$	constant	$\min[s]$	$\min[s - e]$	$\max\left(\frac{w + s}{s}\right)$	(see text)
Decision Mode	Non-preemptive	Preemptive (at time quantum)	Non-preemptive	Preemptive (at arrival)	Non-preemptive	Preemptive (at time quantum)
Throughput	Not emphasized	May be low if quantum is too small	High	High	High	Not emphasized
Response Time	May be high, especially if there is a large variance in process execution times	Provides good response time for short processes	Provides good response time for short processes	Provides good response time	Provides good response time	Not emphasized
Overhead	Minimum	Minimum	Can be high	Can be high	Can be high	Can be high
Effect on Processes	Penalizes short processes; penalizes I/O bound processes	Fair treatment	Penalizes long processes	Penalizes long processes	Good balance	May favor I/O bound processes
Starvation	No	No	Possible	Possible	No	Possible





Comparación de algoritmos

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2





Comparación de algoritmos

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2





Comparación de algoritmos

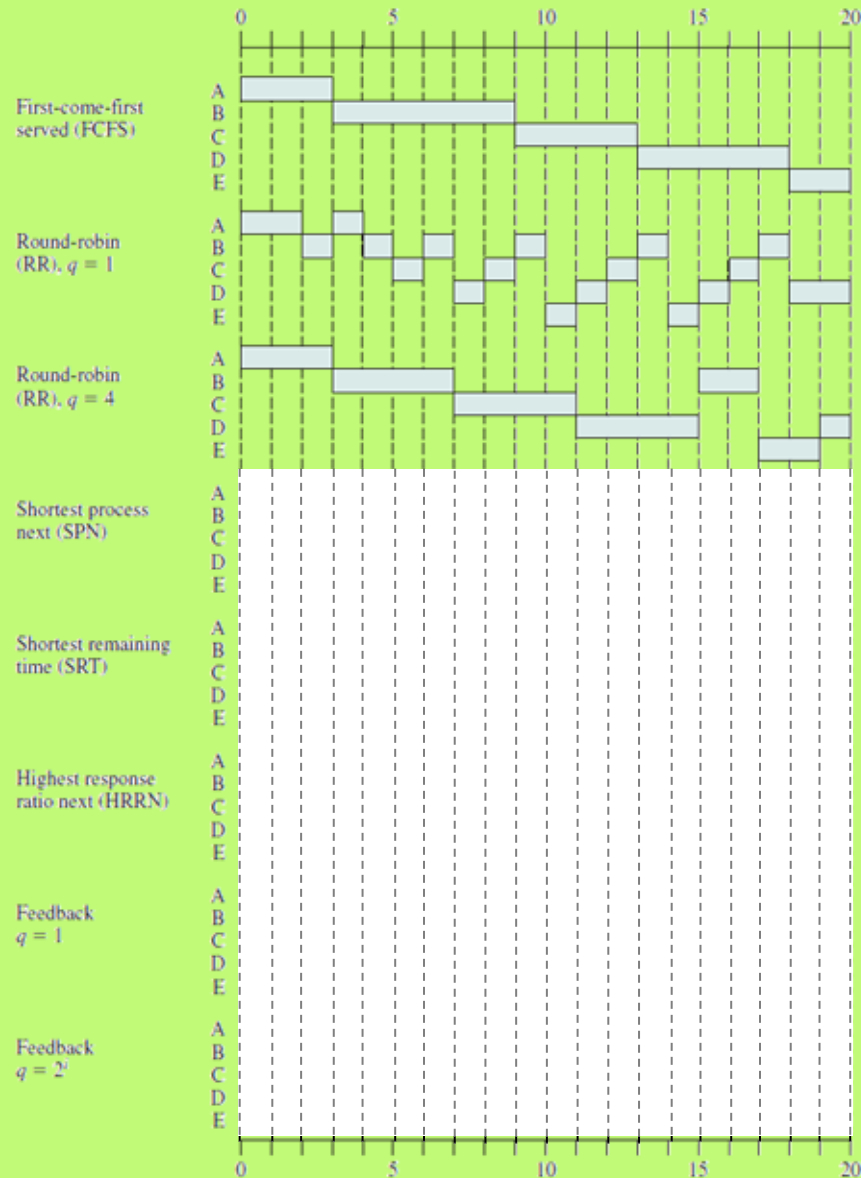
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2





Comparación de algoritmos

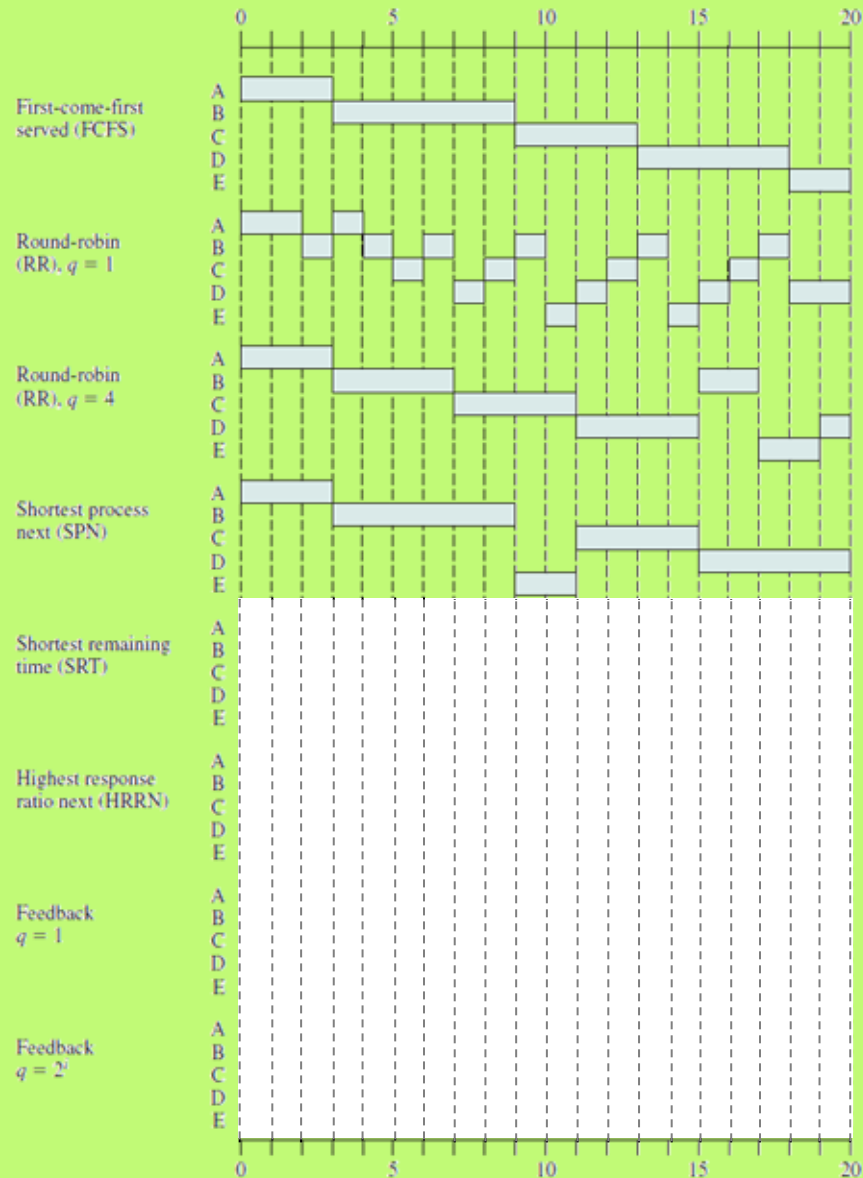
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2





Comparación de algoritmos

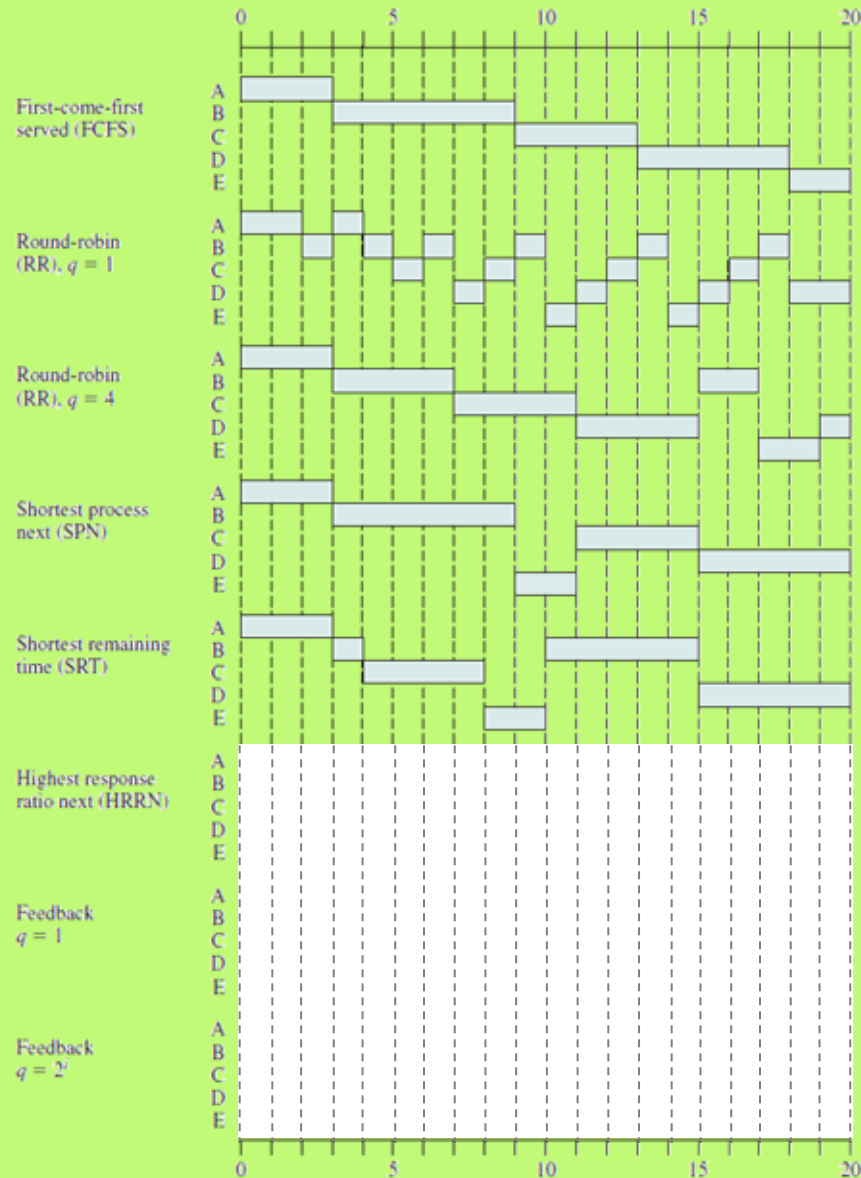
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2





Comparación de algoritmos

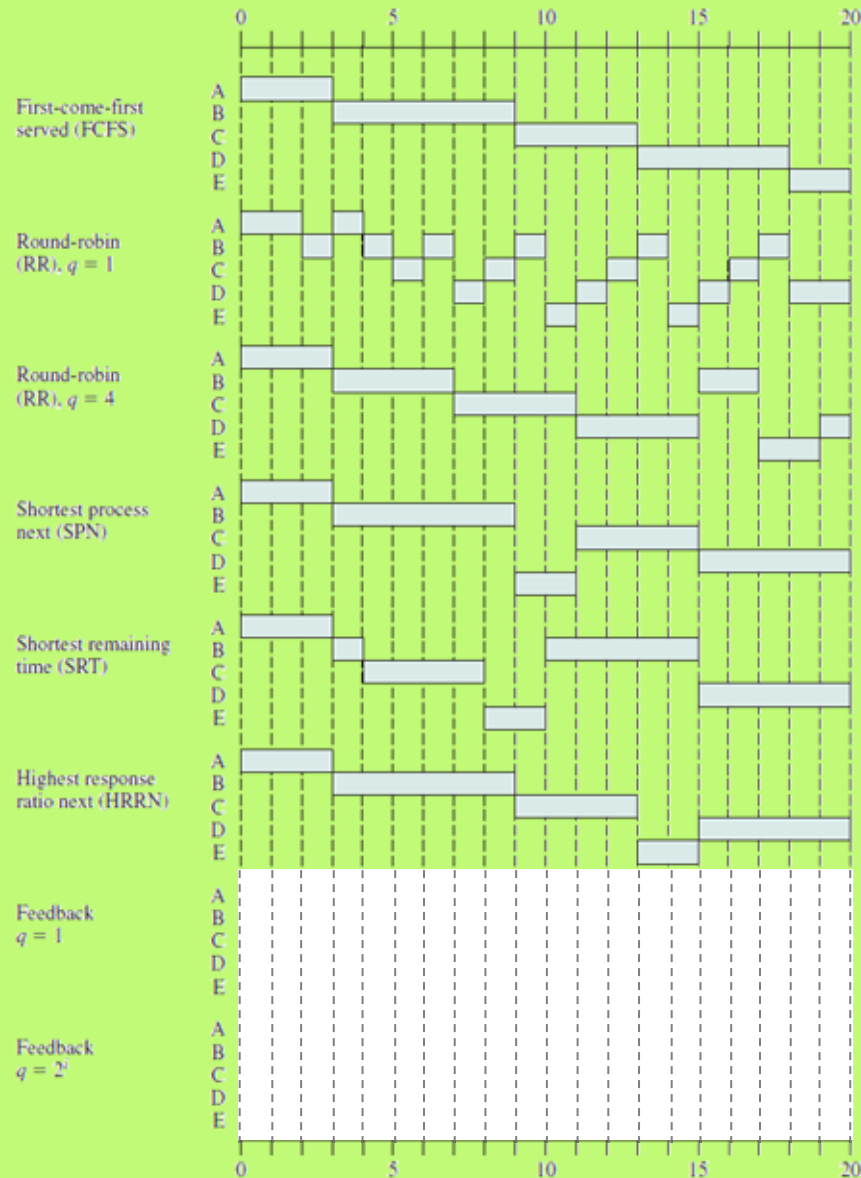
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2





Comparación de algoritmos

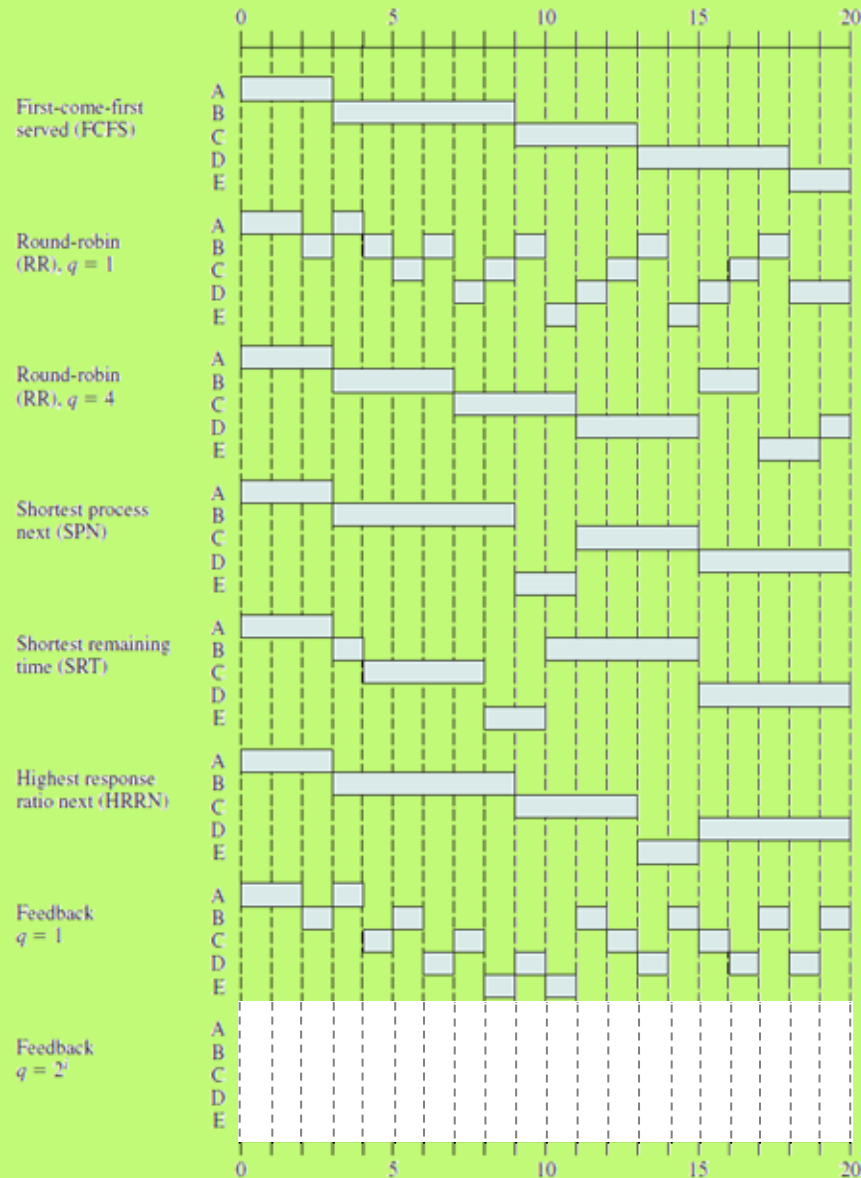
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2





Comparación de algoritmos

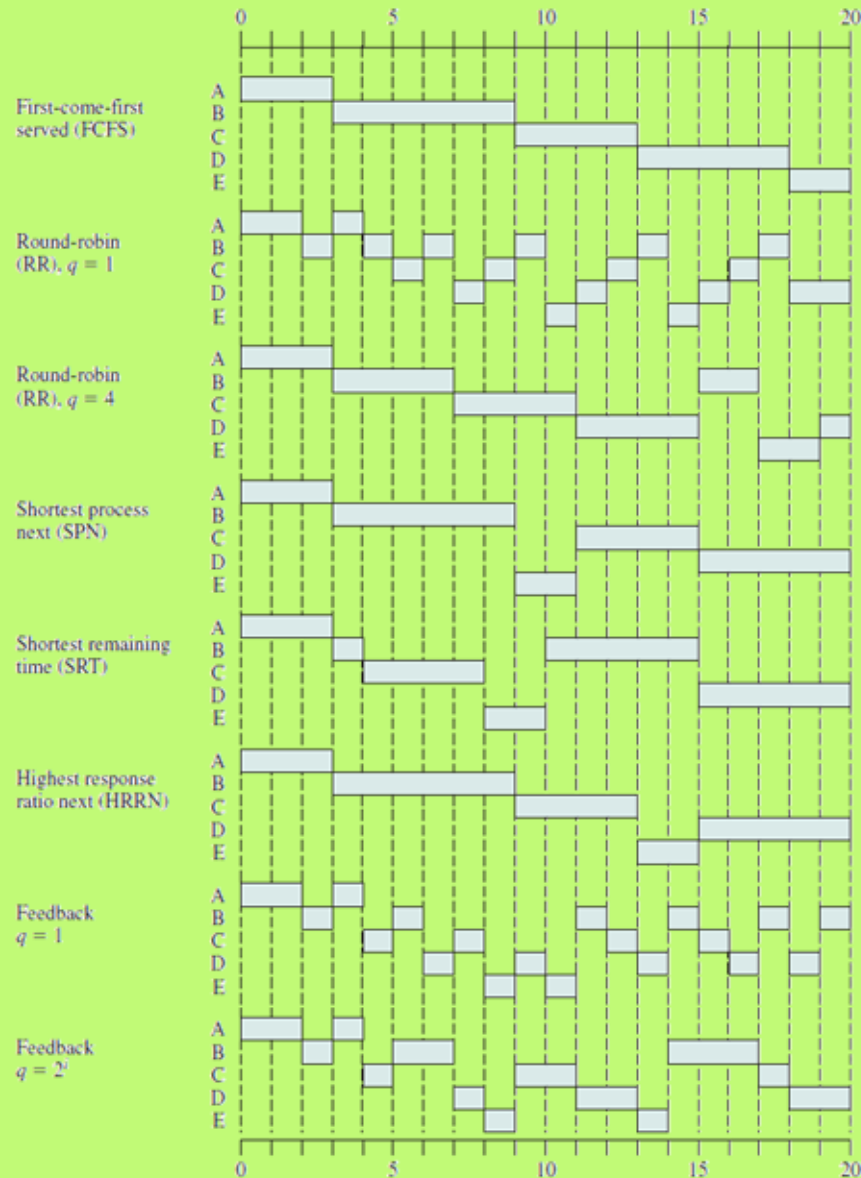
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2





Comparación de algoritmos

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2





Sistemas multiprocesador

- Ya se discutieron dos variantes generales:
 - Multiprocesamiento asimétrico: un procesador lleva la batuta (maneja las instrucciones y recursos del sistema).
 - Multiprocesamiento simétrico (SMP): todos los procesadores se calendarizan tareas.
 - ▶ Es el tipo de multiprocesamiento que proveen los sistemas operativos multipropósito que usamos todos los días.
- Tener más de un procesador permite compartir la carga de trabajo entre procesadores a costa de calendarización más complicada.
 - Por ejemplo, se debe coordinar que dos procesadores no elijan el mismo proceso para ejecutar.





Sistemas multinúcleo



- El sistema operativo los percibe como procesadores lógicos.
- Contribuyen a menor consumo de energía y mayor velocidad de operación.
- Cada núcleo suele contener dos o más hilos de ejecución (***threads de hardware***).
 - Sirven porque el procesador pierde tiempo esperando ciertas operaciones como accesos a memoria principal (fenómeno conocido como ***memory stall***).





Calendarización multinúcleo

- A dos niveles:
 - Sistema operativo calendariza ejecución de *threads* de *software* en *threads* de *hardware*.
 - Cada núcleo calendariza la ejecución de sus *threads* de *hardware*.
- Sistemas multiprocesador se caracterizan por la **frecuencia de sincronización** o **granularización del *multithreading***.
 - Granularización **gruesa**: cada *thread* carga su propia *pipeline* de instrucciones.
 - Granularización **fina**: se intercala ejecución de ciclos de instrucción entre *threads*.





Tabla de granularización

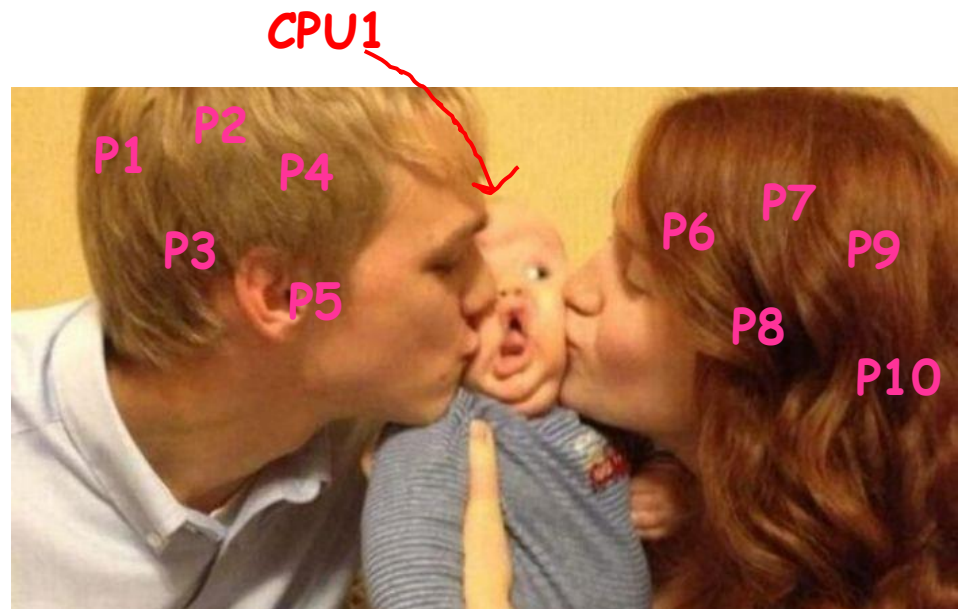
Grain Size	Description	Synchronization Interval (Instructions)
Fine	Parallelism inherent in a single instruction stream	< 20
Medium	Parallel processing or multitasking within a single application	20–200
Coarse	Multiprocessing of concurrent processes in a multiprogramming environment	200–2,000
Very Coarse	Distributed processing across network nodes to form a single computing environment	2,000–1M
Independent	Multiple unrelated processes	Not applicable





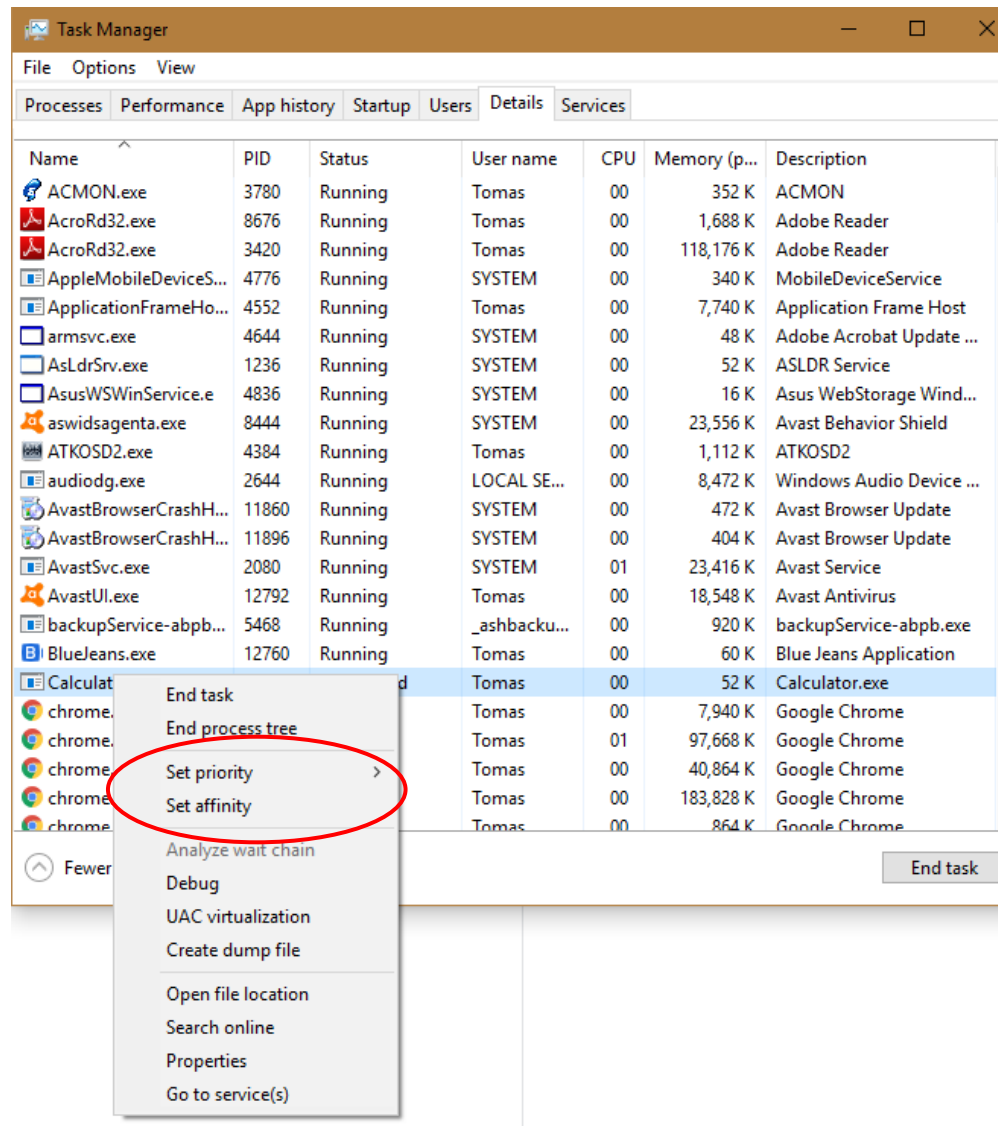
Afinidad por procesador

- ❑ Evitar migración de procesos entre procesadores.
 - ❑ Un proceso “tiene **afinidad**” por el procesador en el que se ejecuta.
 - ❑ Evita el vaciar y repopular frecuentemente las *caché*, para mejorar desempeño.
- ❑ Hay afinidad por niveles.
 - ❑ **Dura**: sistemas que permiten limitar ejecución de procesos a procesadores específicos.
 - ❑ **Suave**: se procura evitarla, pero se permite la migración de procesos.





Afinidad por procesador





Balanceo de carga

- Enemigo natural de la afinidad por procesador.
- Útil en sistemas de multiprocesamiento donde cada procesador tiene una cola privada de procesos (lo común).
- Dos tipos de migración no exclusivos:
 - **Push**: hay un chequeo periódico y (si es necesaria) una redistribución de carga en los procesadores.
 - **Pull**: un procesador inactivo le quita carga a uno ocupado.





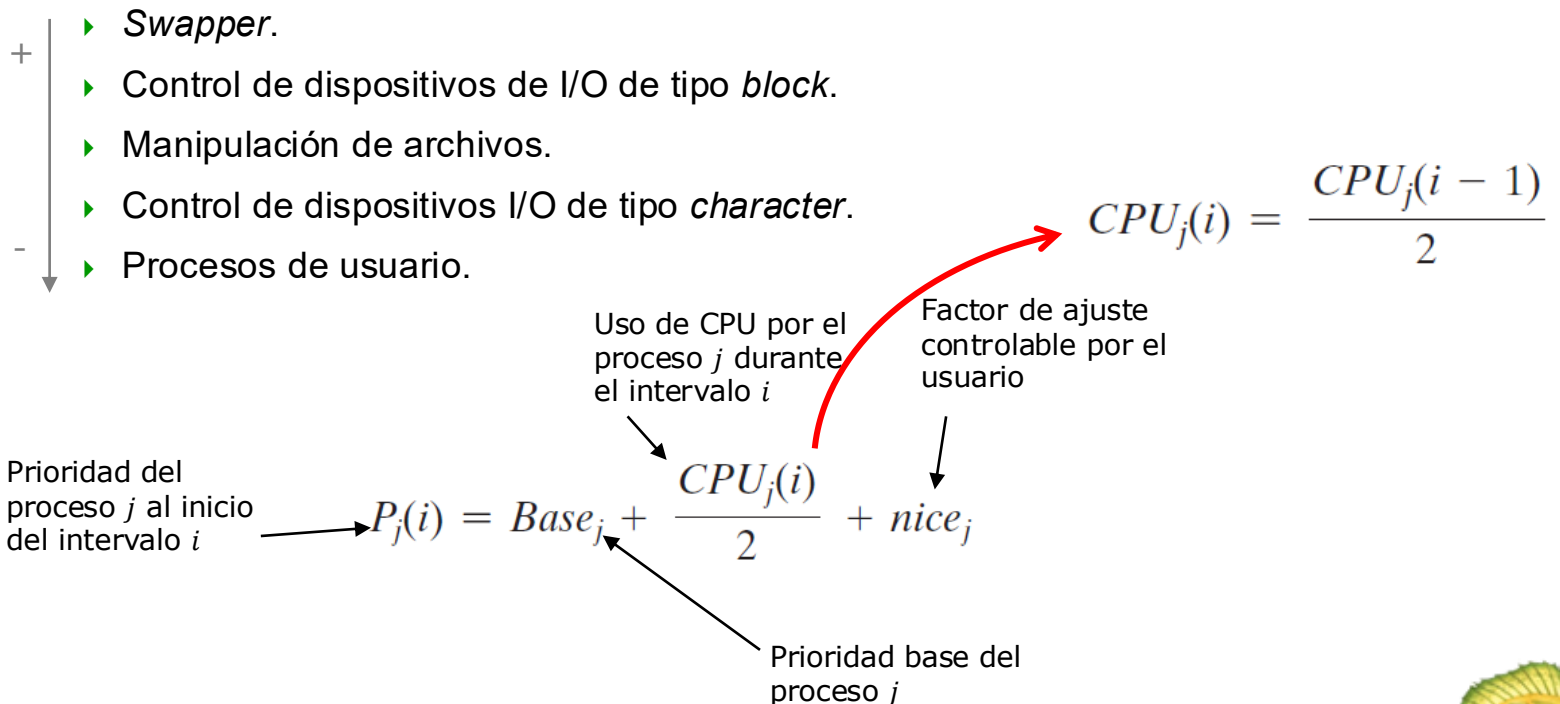
EJEMPLOS





UNIX (tradicional)

- Multicola con retroalimentación, cada cola manejada con *round robin*.
 - *Quantum* de un segundo, al cabo del cual se computa la prioridad de cada proceso y se toma una decisión de calendarización.
 - La prioridad de los procesos varía en rangos o **bandas** determinados por la prioridad de cada cola (o **prioridad base**).





Linux 2.6 (CFS)

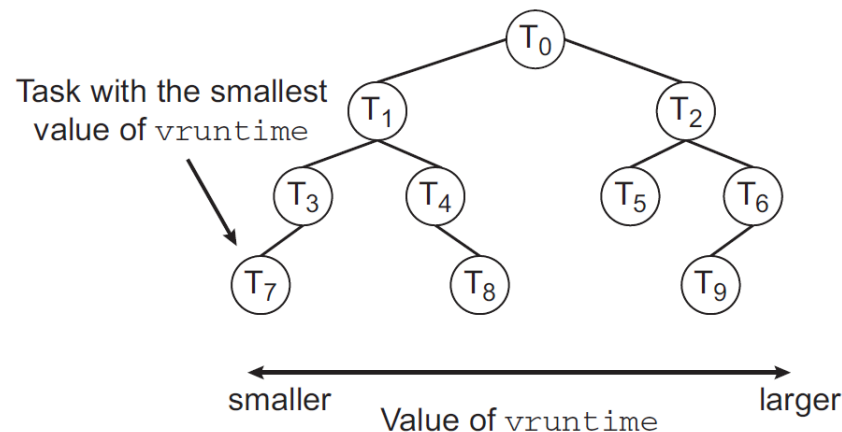
- *Completely Fair Scheduler*: para calendarización de procesos que no necesitan ejecución en tiempo real.
 - Linux separa su calendarización en clases. La calendarización para procesos de tiempo real es diferente.
 - ▶ Básicamente, es multicola con retroalimentación.
- Se establece una **latencia objetivo**: intervalo de tiempo durante el cual todas las tareas ejecutables deberían correr al menos una vez.
 - El período de latencia objetivo se distribuye entre procesos listos, con base en sus prioridades relativas o **nice values**.
 - ▶ Mayor *nice value* corresponde a menor prioridad y, por tanto, menor proporción de CPU.
 - Puede variar para ajustarse al número de tareas en el sistema.





Linux 2.6 (CFS)

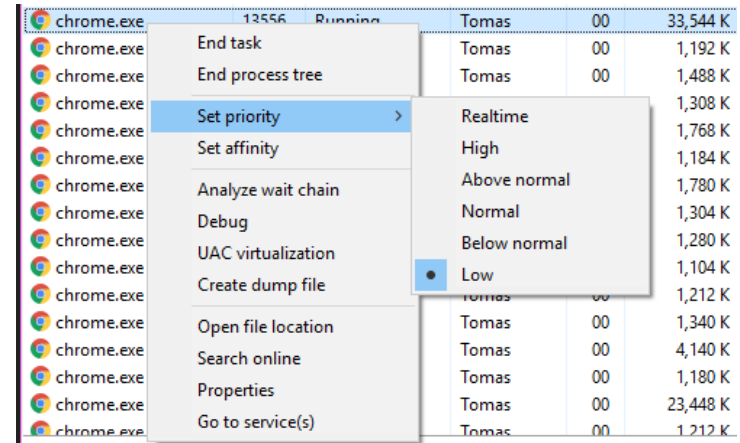
- Se calcula un **tiempo de ejecución virtual** por proceso dependiente del tiempo que éste ha usado el CPU.
 - Periódicamente se acumula el tiempo de ejecución real multiplicado por un factor de peso.
 - ▶ El **peso** de un proceso es aproximadamente $\frac{1024}{1.25^{\text{nice}}}$.
 - ▶ El factor de peso se calcula como una proporción entre el peso del proceso y el de los demás procesos compitiendo por CPU.
- Procesos se ordenan en un **árbol red-black** por `vruntime`.
 - Se mantiene un puntero a la ubicación del nodo en el extremo izquierdo del árbol, que contiene al proceso que tomará el CPU.





Windows

- El calendarizador es llamado *dispatcher*. La calendarización es *preemptive*, multicola con retroalimentación.
- Se manejan 32 niveles de prioridad, ordenados ascendentemente del 0 al 31.
- Las prioridades se organizan en clases y subclases. Las clases son **variable** y **tiempo real**. Las subclases son, ascendiendo por prioridad:
 - ▶ **IDLE_PRIORITY_CLASS**
 - ▶ **BELOW_NORMAL_PRIORITY**
 - ▶ **NORMAL_PRIORITY_CLASS**
 - ▶ **ABOVE_NORMAL_PRIORITY_CLASS**
 - ▶ **HIGH_PRIORITY_CLASS**
 - ▶ **REALTIME_PRIORITY_CLASS**





Windows

- Todas las subclases excepto `REALTIME_PRIORITY_CLASS` pertenecen a la clase variable. Permite cambios de prioridad.
- Cada subclase tiene niveles de **prioridad relativa** para los hilos de ejecución del *kernel* (lo veremos más adelante):

- **IDLE**
- **LOWEST**
- **BELOW_NORMAL**
- **NORMAL**
- **ABOVE_NORMAL**
- **HIGHEST**
- **TIME_CRITICAL**

	real-time	high	above normal	normal	below normal	idle priority
time-critical	31	15	15	15	15	15
highest	26	15	12	10	8	6
above normal	25	14	11	9	7	5
normal	24	13	10	8	6	4
below normal	23	12	9	7	5	3
lowest	22	11	8	6	4	2
idle	16	1	1	1	1	1

- Cada proceso tiene una **prioridad base**, que es la prioridad relativa normal de la clase a la que pertenece.
 - Es la prioridad inicial para cada *thread* de ese proceso.





Windows

- Los *threads* de clase variable reciben **boosts** de prioridad cuando salen del estado de espera. Los *boosts* varían por motivo de espera:
 - E.g., I/O de teclado recibe mayor *boost* que I/O de disco.
 - Procesos (*threads*) CPU-bound reciben ciclos de CPU que no sean usados por procesos I/O-bound. Los dispositivos de I/O se mantienen ocupados.
- Hay una cola por cada prioridad, recorrida en *round robin* sólo si las colas de mayor prioridad están vacías.
 - Cuando un *thread* consume el *quantum* asignado a su cola, su prioridad baja.
 - Se triplica el *quantum* del proceso en el **foreground**, i.e., el proceso seleccionado en pantalla; y también se *boostea* su prioridad.
 - ▶ Mejora tiempo de respuesta para proceso interactuando con el usuario.
 - ▶ Los demás procesos se ejecutan en el **background**.
 - Un *thread* también es *preemptado* cuando otro de mayor prioridad entra a la *ready queue*.





Windows

- ❑ El Windows API permite cambiar las clases de prioridad por proceso y las prioridades relativas por *thread*.
 - ❑ `SetPriorityClass()`
 - ❑ `SetThreadPriority()`
- ❑ Por medio de bibliotecas se permite el manejo de *threads* de usuario.
 - ❑ Se encargan de la calendarización de *threads* de usuario en *user space*.
 - ▶ Más eficiente, por requerir menos llamadas a sistema.
 - ❑ Veremos este tema más adelante.
- ❑ Emplea un sistema de afinidad suave configurable a dura.





Fuentes

- ❑ <https://docs.microsoft.com/en-us/windows/desktop/procthread/scheduling>
- ❑ <http://www.informit.com/articles/article.aspx?p=101760>
- ❑ <https://oakbytes.wordpress.com/2012/07/03/linux-scheduler-cfs-and-virtual-run-time/>
- ❑ <https://www.alouche.net/2012/02/04/linux-cfs-algorithm-and-virtual-runtime/>
- ❑ http://www.qnx.com/developers/docs/6.5.0/index.jsp?topic=%2Fcom.qnx.doc.neutrino_sys_arch%2Fchar.html
- ❑ Silberschatz, A., Galvin, P., & Gagne, G. (2013). *Operating System Concepts*. John Wiley & Sons, Inc.
- ❑ Tanenbaum, A., & Woodhull, A. (2006). *Operating Systems: Design and Implementation*. Prentice Hall.
- ❑ Stallings, W. (2012). *Operating Systems: Internals and Design Principles*. New Jersey: Prentice Hall.

